

Dental Clinic Knowledge Base — Document Lifecycle Sync

Maintainer Guide for `13_pinecone_update_delete_add_ENHANCED.json`

n8n workflow · Google Drive → Pinecone · new / updated / deleted document handling

Purpose: Keep the Pinecone knowledge base used by the dental AI assistant in sync with a Google Drive folder — automatically ingesting new documents, re-indexing updated ones, and removing vectors for deleted ones.

1. What this workflow does

This is an n8n workflow with **four independent entry points** that all feed the same document-processing pipeline:

Entry point	Trigger	What happens
Manual batch ingest	Run KB Ingestion (manual)	Lists every file in the watched Drive folder and (re)processes all of them. Useful for first-time setup or a full re-sync.
New file added	New File Added (Drive Trigger — fileCreated)	Processes only the newly added file.
File updated	File Updated (Drive Trigger — fileUpdated)	Deletes the file's existing vectors first, then reprocesses it — so edits never create duplicate/stale vectors.
File deleted	Check For Deleted Files (hourly schedule)	Compares the current Drive folder contents against a tracking sheet; any file that's disappeared has its vectors removed from Pinecone.

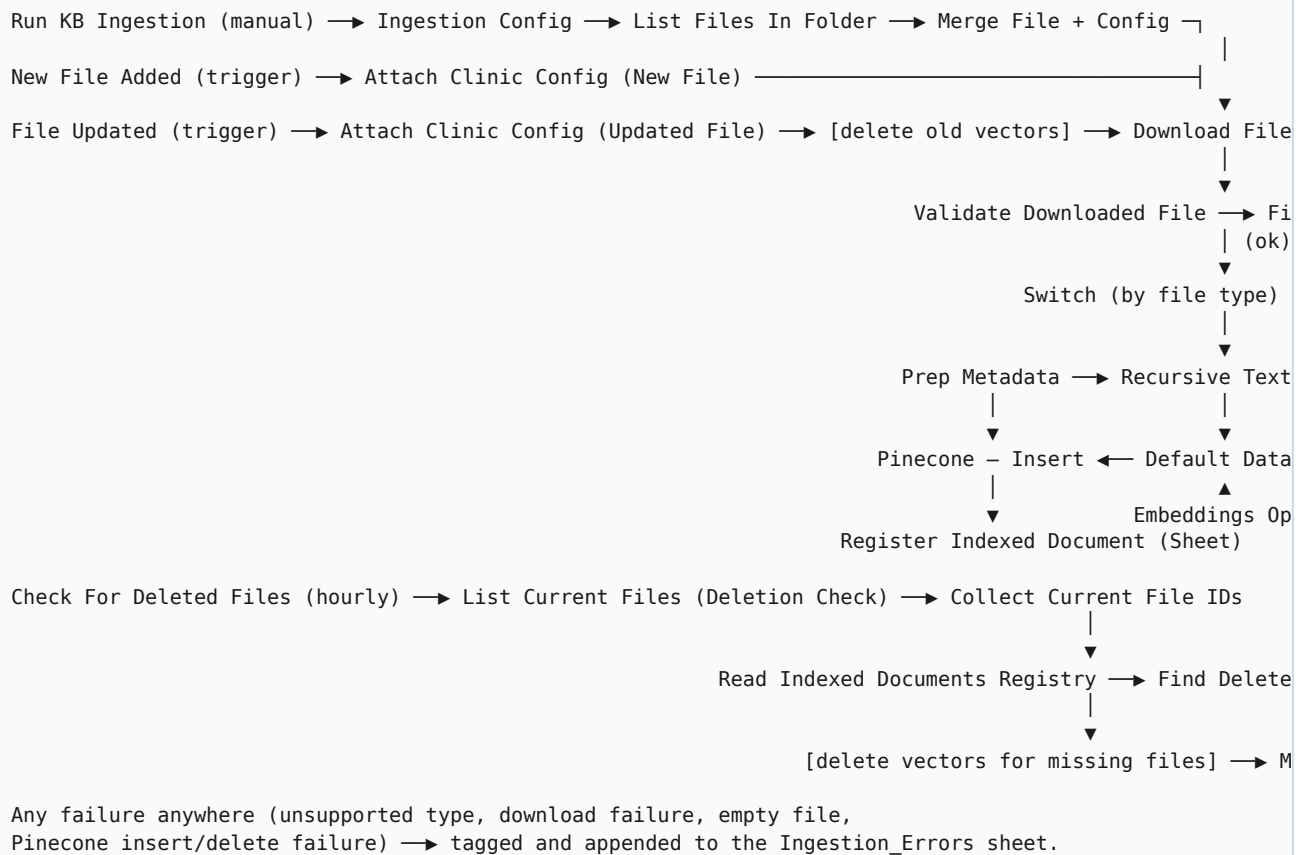
Supported file types: **PDF, DOCX, TXT, Google Docs, CSV, Excel** (routed by the `Switch` node; anything else is logged as unsupported, not silently dropped).

Every vector stored in Pinecone carries this metadata:

```
{
  "clinic_id": "",
  "clinic_name": "",
  "category": "",
  "source_document": "",
  "google_drive_file_id": "",
  "file_type": "",
  "last_updated": ""
}
```

`google_drive_file_id` is the key everything else depends on — it's how the workflow finds and removes the right vectors when a file changes or disappears.

2. Architecture at a glance



Key design decision — why config is "attached" separately in each branch: `Prep Metadata` and `Pinecone - Insert` read clinic/Pinecone settings from *the current item*, not from a hardcoded reference to one specific node. That's what lets all four entry points share the exact same download → convert → embed → insert pipeline without duplicating it four times. If you add a fifth entry point later, just make sure it merges the same config fields onto its item (`clinic_id` , `clinic_name` , `category` , `pinecone_index` , `pinecone_namespace` , `pinecone_host` , `registry_sheet_id` , `embedding_dimension`) before it reaches `Download File` .

3. Why deletion needs two Pinecone API calls, not one

n8n's built-in Pinecone node (the LangChain vector store node) only supports **insert** and **load/retrieve** — there is no delete operation in the node itself. So deletion is done with two `HTTP Request` nodes calling Pinecone's REST API directly.

The non-obvious part: you might expect a single call —

```
POST {host}/vectors/delete
{ "filter": { "google_drive_file_id": { "$eq": "<file_id>" } } }
```

— but **Pinecone serverless indexes do not support deleting by metadata filter**. Only pod-based (legacy) indexes do. Since serverless is the default for new Pinecone projects, this workflow uses a

pattern that works on **both**:

1. **Find Vector IDs** — `POST {host}/query` with the metadata filter and a dummy all-zero vector (required by the endpoint, ignored for filtering purposes) → returns the IDs of every matching vector.
2. **Delete By ID** — `POST {host}/vectors/delete` with those explicit IDs.

If you ever migrate to a pod-based index and want to simplify this, you *can* collapse it back to a single filtered delete call — but the two-step version will keep working regardless, so there's no need to change it.

This pattern appears twice in the workflow — once under "**Delete Old Vectors**" (update flow) and once under "**Delete Removed File Vectors**" (deletion flow). They're separate node chains rather than one shared chain, because each needs to route its "done" output somewhere different afterward (reprocess vs. mark-as-deleted) — n8n can't conditionally route based on which trigger a shared node came from without extra branching, so duplicating this small chain keeps the workflow easier to read.

4. Why deletion detection is a scheduled diff, not a Drive trigger

Google Drive has no "file deleted" trigger event in n8n. The two realistic options were:

- **Drive Changes API** (real-time-ish): more "native," but requires persisting a `pageToken` between runs, handling first-run bootstrapping, and renewing watch channels — meaningfully more complex for this use case.
- **Scheduled poll + diff against a registry** (what's implemented): every hour, list the current folder contents, compare against a sheet of "files we know we indexed," and treat anything missing as deleted.

The registry sheet (`Indexed_Documents`) also gives you an audit trail of what's currently indexed, which is useful on its own for troubleshooting ("why is the bot citing a document that doesn't exist anymore?").

Trade-off to know about: deletions are caught within an hour, not instantly. If you need faster detection, lower the schedule interval on the `Check For Deleted Files` node — but weigh that against extra Drive/Sheets API calls.

5. One-time setup checklist

5.1 Pinecone

- ☐ Note your index's **host URL** from the Pinecone console (Index → Overview). Looks like `https://smiley-dental-clinic-kb-xxxx.svc.us-east-1-aws.pinecone.io`.
- ☐ Confirm your embedding model's dimension (1536 for `text-embedding-3-small`, already used by `Embeddings OpenAI (KB)`).

5.2 Credentials

- ☐ **Google Drive OAuth2** — already exists (reused from the original workflow); make sure it's selected on the two new Drive Trigger nodes and the new Drive search node.

- ☐ **Google Sheets OAuth2** — create/select on all Sheets nodes (5 of them).
- ☐ **Pinecone Data API (Header Auth)** — create a new **Generic Credential** → **Header Auth** credential:
 - ☐ Header name: `Api-Key`
 - ☐ Header value: your Pinecone API key (same key as your existing Pinecone credential)

(Not the built-in "Pinecone API" credential type — it has known authentication bugs when used from the HTTP Request node.)

5.3 Google Sheet — create one spreadsheet with two tabs

Indexed_Documents (the registry):

Column	Notes
google_drive_file_id	key column — used for matching/updating
source_document	filename
category	e.g. "Policy and FAQ"
file_type	pdf / docx / txt / etc.
clinic_id	for multi-clinic setups
clinic_name	
last_indexed	ISO timestamp
status	<code>Active</code> or <code>Deleted</code>

Ingestion_Errors (the error log):

Column	Notes
timestamp	
google_drive_file_id	
source_document	
error_type	<code>download_failed</code> / <code>empty_file</code> / <code>unsupported_file_type</code> / <code>pinecone_insert_failed</code> / <code>pinecone_delete_failed</code>
error_message	
stage	which step failed

5.4 Fill in config values (must match across 4 nodes)

Open **each** of these nodes and set the same values for `pinecone_host`, `registry_sheet_id`, and `embedding_dimension`:

- `Ingestion Config`

- Attach Clinic Config (New File)
- Attach Clinic Config (Updated File)
- Attach Clinic Config (Deletion Check)

This is the #1 thing to check if something isn't working. These four nodes hold duplicated literal values because n8n trigger nodes can't reference another node's output. If you change the Pinecone index, host, or registry sheet, update it in all four places.

5.5 Folder ID

The Drive folder ID is hardcoded in **three** places: `Ingestion Config`, `New File Added`, and `File Updated`. If the watched folder ever changes, update all three.

6. Testing before going live

1. Run `Run KB Ingestion` manually first — this does a full batch ingest and populates `Indexed_Documents` for every existing file. Do this before turning on the scheduled deletion check, otherwise the very first scheduled run will think every existing file was "deleted" (nothing in the registry yet to compare against).
2. Add a test file to the Drive folder → confirm the `New File Added` trigger fires, the file appears in `Indexed_Documents`, and you can retrieve it via a FAQ query in the main assistant.
3. Edit that test file's content → confirm `File Updated` fires, old vectors are gone (check the doc count in Pinecone before/after, or query for old content and confirm it's no longer returned), and new content is retrievable.
4. Delete the test file from Drive → wait for the next hourly run (or manually execute `Check For Deleted Files`) → confirm its `Indexed_Documents` row flips to `status = Deleted` and it's no longer retrievable.
5. Upload an unsupported file type (e.g. a `.zip`) → confirm it shows up in `Ingestion_Errors` with `error_type = unsupported_file_type`, not silently ignored.
6. Upload a 0-byte file → confirm it's logged as `empty_file`.

7. Known limitations

- **Deletion detection has up to a 1-hour delay** (schedule interval) — see §4.
- **"Empty document" detection only catches 0-byte files.** A file that downloads fine but has no *extractable* text (e.g. a scanned image PDF with no OCR text layer) won't be caught by this check — it happens inside the document-loading step, downstream of where this check runs. If that becomes a real problem, an explicit text-extraction-and-length-check step can be added before the Pinecone insert step.
- **Drive Trigger nodes poll every minute** rather than pushing instantly — fine for this use case, just don't expect true real-time.
- **Node parameter shapes drift between n8n versions**, especially for `Switch` node fallback options and Drive Trigger parameters. If something doesn't validate on import, check those two node types

first.

- **If you migrate Pinecone indexes** (e.g., serverless → pod-based, or a new index name), update `pinecone_index` / `pinecone_host` in all four config-holding nodes (§5.4), and re-run a full batch ingest.

8. Quick node reference

Node	Type	Role
Run KB Ingestion	Manual Trigger	Kick off a full batch re-sync
Ingestion Config	Set	Canonical config for the batch path
List Files In Folder	Google Drive	Lists all files for batch processing
Merge File + Config	Code	Re-attaches config onto each batch file item
New File Added / File Updated	Google Drive Trigger	Event-driven entry points
Attach Clinic Config (...)	Set	Config for each trigger-driven entry point
Delete Old Vectors — Find Vector IDs / Extract IDs / Any Vectors? / Delete By ID	HTTP Request / Code / IF / HTTP Request	Removes stale vectors before reprocessing an updated file
Download File	Google Drive	Downloads the file's binary content
Validate Downloaded File / File Valid?	Code / Switch	Catches failed downloads and empty files
Switch	Switch	Routes by file type; unsupported types go to the error log
Prep Metadata	Code	Builds the metadata object attached to every chunk
Recursive Text Splitter	LangChain Text Splitter	Chunks the document
Default Data Loader	LangChain Document Loader	Extracts text + attaches metadata per chunk
Embeddings OpenAI (KB)	LangChain Embeddings	Generates vector embeddings
Pinecone — Insert	LangChain Vector Store	Writes vectors to Pinecone
Register Indexed Document	Google Sheets	Records/updates the file's registry row
Check For Deleted Files	Schedule Trigger	Hourly deletion check
List Current Files (Deletion Check)	Google Drive	Lightweight listing — does NOT reprocess files
Collect Current File IDs	Code	Builds the "what's currently in Drive" list
Read Indexed Documents Registry	Google Sheets	Reads what we think is indexed
Find Deleted Files	Code	Diff's the two lists

Delete Removed File Vectors — ...	HTTP Request / Code / IF / HTTP Request	Same delete pattern as the update flow
Mark Registry Row As Deleted	Google Sheets	Marks the registry row Deleted
Tag Error — ... (5 nodes)	Code	Labels each failure source before logging
Log Ingestion Error	Google Sheets	Shared error sink

Generated as part of the dental clinic AI assistant project. Keep this document alongside the workflow JSON when handing off maintenance.